

AIDA vs the AXI ecosystem (tasks-axi / firstmate / no-mistakes)

AIDA positioning · best-effort comparison — live source:
github.com/joemooney/aida

rendered 2026-07-10

One focused comparison, in the docs/positioning/ series. Neighbor: **AXI** — “Agent eXperience Interface”, Kun Chen’s (kunchenguid) family of agent-native tools. Snapshot: 2026-06-28. Paired note: docs/competitive-analysis/2026-06-28-axi-ecosystem.md.

Last updated: 2026-07-09 (doc reviewed). Landscape snapshot: 2026-06-28.

TL;DR

AXI is the **most serious neighbor we’ve found on the *interface* layer**, and the honest read is split:

- **AXI is ahead of us on interface ergonomics**, with benchmarks to back it. Its whole thesis is “treat the agent’s token budget as a first-class constraint,” and AIDA’s CLI does the opposite (emoji, tables, human prose).
- **AIDA is ahead on the *substrate*** — a git-canonical typed requirement graph with stable IDs, code↔spec traces, and an enforced lifecycle. The AXI tools (and the Beads model tasks-axi borrows) have a dependency graph but no durable spec identity, no code traceability, no lifecycle state machine.

These are different layers. The right mental model is *not* “AIDA vs AXI” as substitutes — it’s “AIDA should **adopt** AXI’s interface principles on top of its graph substrate.” The threat is asymmetric: it is easier for an AXI tool to grow a graph than for AIDA to retrofit agent-ergonomics, so the clock favors moving first.

What AXI is

A coherent ecosystem, not scattered tools — and with real traction:

Tool	☆	What it is	AIDA analogue
axi	~1.1k	The thesis : 10 design principles for agent-native CLIs + benchmarks vs MCP	— (a philosophy; AIDA has no equivalent manifesto)

Tool	☆	What it is	AIDA analogue
no-mistakes	~3.8k	A git-push proxy — git push no-mistakes → disposable worktree → review/test/docs/lint pipeline → clean PR + CI auto-fix	aida queue work --auto-complete (the drain), delivered as a push interceptor
gnhf	~2.5k	“Good Night Have Fun” — keep agents running long with precise token/iteration/stop caps	the autonomous drain + aida goal
firstmate	~330	“Talk to one agent, ship with a crew” — one liaison agent spawns a worktree-isolated crew in tmux, supervises, escalates only real decisions; opt-in persistent “secondmates” = domain supervisors	the advisor + orchestrator + queue ; secondmates ≈ our SPIKE-10 subsystem advisors
treehouse	~430	“Manage worktrees without managing worktrees”	our isolation:worktree fan-out
tasks-axi	~11	Task/backlog manager for agents — the direct neighbor	aida itself (the backlog/queue surface)
gh-axi	~95	GitHub CLI for agents (token-efficient)	our forge layer (EPIC-35 / STORY-621)
lavish-axi	~1k	“HTML is the new markdown” — opens an agent-generated HTML artifact in the browser, human pinpoint-annotates elements, agent long-polls + revises	— (a presentation/review layer ; orthogonal to AIDA's substrate. Building it = the surface-complexity the Trojan-horse positioning rejects)

Delivery is itself a design choice: firstmate is “*not a harness, not a CLI — a directory*” (AGENTS.md + skills + bash). tasks-axi ships as an Agent Skill (npx skills add) or a SessionStart hook. Zero-install, agent-loads-on-demand.

The two challenges that matter

1. The interface bet: AXI’s evidence says token-efficient CLI > MCP

AXI’s published benchmarks (Claude Sonnet 4.6, hundreds of runs):

Surface	Success	Avg cost	Turns
gh-axi (token-efficient CLI)	100%	\$0.050	3
gh (human CLI)	86%	\$0.054	3
GitHub MCP	87%	\$0.148	6

This one landed: AXI’s finding replicated. AIDA reproduced the MCP-vs-CLI benchmark on its own surfaces (SPIKE-73: MCP ~2.2× the token-efficient CLI at equal-or-lower success) and re-weighted accordingly — the CLI (with AIDA_AGENT_OUTPUT/TOON) is now the

primary agent surface and MCP the typed *option*, not the center. Competitor investment used as validation, then acted on rather than dismissed.

Scored against AXI's 10 principles at the 2026-06-28 snapshot, AIDA's CLI was roughly **4-5/10** — the #1 output gap has since closed (see below): - **Strong:** pre-computed aggregates (the cache; `aida status`, the queue footer), ambient context (SessionStart hooks, brief-polling), contextual next-step suggestions, consistent help. - **Missing:** #1 token-efficient output (we print emoji/tables; `--json` is verbose JSON, not **TOON**); #8 content-first (bare `aida` shows help, not live data); partial on minimal-schemas and structured-errors.

Note: AIDA's human-facing emoji output is a *deliberate* choice (and the operator likes it). The **agent-facing** gap has since closed: `AIDA_AGENT_OUTPUT` forces agent mode and `aida list/show/queue list/status` render as TOON (TASK-970/TASK-964) — a token-optimized mode for when an agent, not a human, is the caller. The emoji human path is deliberately unchanged.

2. The backlog bet: tasks-axi is the same arena, different substrate

tasks-axi is “task and backlog manager for agents.” Its model:

- A hand-editable **backlog.md as source of truth**, byte-exact round-trip — Karpathy-style “structured markdown queryable by Claude,” which our own CLAUDE.md explicitly names *the floor* AIDA rises above.
- **Borrows the dependency-graph + ready-query model from beads** — i.e. our nearest competitor, and gastownhall = the Gas Town that went cross-vendor in the 2026-06-26 snapshot. The neighbor cluster is consolidating around a shared model.
- Token-efficient output, idempotent mutations, contextual next-steps, a session hook that feeds the live backlog into every agent session.

Where AIDA is genuinely differentiated (and tasks-axi/Beads are not): - **Stable spec IDs** that survive edits and reorderings (markdown line-items do not). - **Code↔spec traces** (`// trace:STORY-706`) — a bidirectional link between intent and implementation. No markdown backlog has this. - **A typed relationship graph** (parent/child/blocks/references) with transitive queries, not just a flat dependency list. - **An enforced lifecycle** (Draft→Approved→...→Completed→Released) with merge-driven auto-promotion and role gates. - **Git-canonical history** — every state change is a commit; the backlog *is* the audit trail.

tasks-axi's explicit virtue — “the markdown stays the source of truth” — is also its ceiling: markdown can't carry stable identity, typed edges, or a lifecycle without becoming the thing AIDA already is.

Where each wins

Dimension	AXI ecosystem	AIDA
Agent-facing token cost	Ahead on ergonomics polish (benchmarked)	TOON / AIDA_AGENT_OUTPUT shipped (TASK-970/964); emoji human path deliberate
Zero-install / adoption friction	Wins (skill / npx / hook, no binary)	Behind (a Rust binary to build/install)
Orchestration mechanics (worktrees, long-run, PR pipeline, crew)	Mature, modular	Mature, integrated (--auto-complete)
Durable spec substrate (IDs, traces, typed graph, lifecycle)	Absent	Wins — the moat
Cross-session memory of <i>intent</i>	Session/issue-tracker driven	Wins (the graph persists; traces link code→intent)
Integration vs assembly	Assembled point-tools (compose-your-own)	One integrated system

The synthesis (the actual recommendation)

This is not a “beat AXI” situation. The winning position is **AXI ergonomics on top of the AIDA graph**:

1. **Adopt AXI’s output principles for the agent-facing path** — ✓ done: AIDA_AGENT_OUTPUT/TOON shipped (TASK-970/964) with minimal default schemas, agent-mode gating, and the emoji human path untouched.
2. **Reproduce the MCP-vs-CLI benchmark on AIDA’s own surfaces** — ✓ done (SPIKE-73): MCP $\sim 2.2\times$ the CLI at equal-or-lower success, so the token-efficient CLI is now the primary agent surface and MCP the typed option, not the center.
3. **Keep investing in the substrate** — the remaining live recommendation: it’s the one thing the whole neighbor cluster (AXI + Beads + Gas Town) lacks, and the hardest to copy.

Tripwires

- **A neighbor grows a real graph.** If tasks-axi/Beads adds stable IDs + code traces + a lifecycle, it becomes AXI-ergonomic *and* graph-backed — strictly ahead of today’s AIDA. This is the one to watch; it’s a smaller step for them than agent-ergonomics is for us.
- **AXI principles become table stakes.** If “token-efficient agent output” becomes the expected norm, AIDA’s human-formatted CLI reads as dated to agents.
- **The skill/hook distribution model wins adoption.** Zero-install npx skills add is a lower-friction funnel than “build the Rust binary.”

See also

- docs/competitive-analysis/2026-06-28-axi-ecosystem.md — the dated note + ecosystem map
- docs/positioning/vs-karpathy-md.md — the markdown-as-floor argument (tasks-axi is its agent-ergonomic incarnation)
- The AXI 10 principles: <https://github.com/kunchenguid/axi>